

VLLM专题（十八）—离线推理



大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续

👑 您已是超级会员，正在免费阅读会员专享内容

[查看更多超级会员权益](#) >

您可以在自己的代码中使用 vLLM 对一组提示（prompts）进行推理。

离线 API 基于 `LLM` 类。要初始化 vLLM 引擎，需要创建一个 `LLM` 的新实例，并指定要运行的模型。

例如，以下代码从 HuggingFace 下载 `facebook/opt-125m` 模型，并使用默认配置在 vLLM 中运行它：

```
llm = LLM(model="facebook/opt-125m")
```

在初始化 `LLM` 实例后，您可以使用多种 API 执行模型推理。可用的 API 取决于正在运行的模型类型：

- **生成式模型（Generative models）**：输出 logprobs（对数概率），并通过采样获得最终的输出文本。
- **池化模型（Pooling models）**：直接输出其隐藏状态（hidden states）。

有关每个 API 的更多详细信息，请参考上述页面。

1. 配置选项

本节列出了运行 vLLM 引擎时最常用的配置选项。如需完整列表，请参阅 **Engine Arguments** 页面。

1.1 模型解析（Model Resolution）

vLLM 通过检查模型仓库中 `config.json` 文件的 `architectures` 字段，并找到与 vLLM 注册的对应实现来加载与 HuggingFace 兼容的模型。然而，在以下情况下，我们的模型解析可能会失败：

1. 模型仓库的 `config.json` 文件缺少 `architectures` 字段。
2. 非官方仓库使用未在 vLLM 中记录的替代名称引用模型。
3. 相同的架构名称被用于多个模型，导致加载模型时产生歧义。

为了解决这些问题，可以通过向 `hf_overrides` 选项传递 `config.json` 的覆盖参数来显式指定模型架构。例如：

```
model = LLM(  
    model="cerebras/Cerebras-GPT-1.3B",  
    hf_overrides={  
        "architectures": ["GPT2LMHeadModel"]}, # GPT-2  
)
```

我们的 [支持模型列表](#) 展示了 vLLM 能够识别的模型架构。

2. 减少内存使用（Reducing Memory Usage）

大型模型可能会导致您的机器内存不足（Out of Memory, OOM）。以下是一些有助于缓解此问题的选项：

2.1 张量并行（Tensor Parallelism, TP）

张量并行（通过 `tensor_parallel_size` 选项配置）可以将模型拆分到多个 GPU 上运行，从而分摊计算负载和内存占用。

以下代码将模型拆分到 2 个 GPU 上运行：

```
llm = LLM(model="ibm-granite/granite-3.1-8b-instruct",  
          tensor_parallel_size=2)
```

重要提示

为了确保 vLLM 正确初始化 CUDA，您应该避免在初始化 vLLM 之前调用相关函数（例如 `torch.cuda.set_device()`）。否则，您可能会遇到类似“RuntimeError: Cannot re-initialize CUDA in forked subprocess”的错误。

要控制使用哪些设备，请改为设置 `CUDA_VISIBLE_DEVICES` 环境变量。

2.2 量化

量化模型在降低精度的情况下占用更少的内存。

静态量化模型可以从 HF Hub 下载（一些流行的模型可以在 Neural Magic 上找到），并且可以直接使用，无需额外配置。动态量化也通过量化选项得到支持。

2.3 上下文长度和批量大小

通过限制模型的上下文长度（`max_model_len` 选项）和最大批量大小（`max_num_seqs` 选项），您可以进一步减少内存使用。

```
llm = LLM(model="adept/fuyu-8b",  
          max_model_len=2048,  
          max_num_seqs=2)
```

3. 性能优化与调优

通过微调各种选项，您有可能提高 vLLM 的性能。